

Advanced Analysis of CRC and Cyclic Code Behavior

Your briefing is mathematically precise. Below is a structured, visual, and conceptually deep reinforcement of the key ideas — focusing on *why* CRC works and *exactly when* it fails.

1 What Really Happens at the Receiver

Polynomial View of Transmission

We model everything in **GF(2)** (binary arithmetic, XOR operations).

$$\begin{aligned} [\\ R(x) &= C(x) + E(x) \\] \end{aligned}$$

Where:

- **C(x)** → original transmitted codeword
- **E(x)** → error polynomial
- **R(x)** → received polynomial

Since CRC constructs codewords such that:

$$\begin{aligned} [\\ C(x) \bmod g(x) &= 0 \\] \end{aligned}$$

Then at the receiver:

$$\begin{aligned} [\\ R(x) \bmod g(x) &= E(x) \bmod g(x) \\] \end{aligned}$$

Key Insight

The syndrome depends **only on the error polynomial**, not on the original data.

2 When Does CRC Fail?

CRC fails if and only if:

$$\begin{bmatrix} E(x) \text{ is divisible by } g(x) \\ \end{bmatrix}$$

That means:

$$\begin{bmatrix} E(x) = g(x) \cdot Q(x) \\ \end{bmatrix}$$

If this happens:

$$\begin{bmatrix} E(x) \bmod g(x) = 0 \\ \end{bmatrix}$$

So the syndrome = 0 $\rightarrow$ receiver falsely accepts corrupted data.

3 Visualizing the Failure Condition

Dataword $d(x)$: $1001 = x^3 + 1$
 Appending three 0's. Multiplying by x^3 .
 We get $x^6 + x^3 = 1001000$
 Generator $g(x) = x + 1$

$$\begin{array}{r}
 x^5 + x^4 + x^3 \\
 x+1 \overline{) x^6 + x^3} \\
 \underline{x^6 + x^5} \\
 x^5 + x^3 \\
 \underline{x^5 + x^4} \\
 x^4 + x^3 \\
 \underline{x^4 + x^3} \\
 0 \text{ Remainder}
 \end{array}$$

Codeword $c(x) = \text{Dataword} + \text{Remainder}$
 $= 1001000 = x^6 + x^3$

Lets say, single bit error $e(x) = x^5$
 i.e. 6th LSB is inverted: 0100000. It
 can be seen that $e(x)$ is not divisible
 by $g(x)$ i.e. the error is caught.

$$\begin{array}{r}
 x^4 + x^3 + x^2 + x^1 + 1 \\
 x+1 \overline{) x^5} \\
 \underline{x^5 + x^4} \\
 x^4 \\
 \underline{x^4 + x^3} \\
 x^3 \\
 \underline{x^3 + x^2} \\
 x^2 \\
 \underline{x^2 + x^1} \\
 x \\
 \underline{x + 1} \\
 0 \text{ Remainder not zero,} \\
 \text{thus error occurred} \\
 \text{and is caught.}
 \end{array}$$

It can also be seen that distorted
 codeword $c(x) + e(x) = x^6 + x^5 + x^3$
 is also not divisible by $g(x)$ i.e. the
 error is caught

$$\begin{array}{r}
 x^5 + x^2 + x^1 + 1 \\
 x+1 \overline{) x^6 + x^5 + x^3} \\
 \underline{x^6 + x^5} \\
 x^3 \\
 \underline{x^3 + x^2} \\
 x^2 \\
 \underline{x^2 + x^1} \\
 x \\
 \underline{x + 1} \\
 0 \text{ Remainder not zero,} \\
 \text{thus error occurred} \\
 \text{and is caught.}
 \end{array}$$

CRC does **not** check whether bits changed.

It checks whether the received polynomial remains inside the cyclic code space.

If the corrupted word still belongs to the cyclic code space $\rightarrow$ undetectable error.

4 Why Generator Design Matters

The generator polynomial determines what patterns are detectable.

A good generator must eliminate common error patterns from being multiples of $g(x)$.

5 Detection Capabilities Explained Mathematically

5.1 Single-Bit Errors

A single-bit error at position i :

$$\begin{bmatrix} E(x) = x^i \end{bmatrix}$$

For this to be undetectable:

$$\begin{bmatrix} x^i \text{ must be divisible by } g(x) \end{bmatrix}$$

This is impossible if:

- $g(x)$ has at least two terms
- constant term = 1

Thus:

- ✓ CRC detects all single-bit errors
- ✓ Provided $g(x)$ is not a monomial

5.2 Double-Bit Errors

A 2-bit error:

$$\begin{bmatrix} E(x) = x^i + x^j \end{bmatrix}$$

Factor form:

$$\begin{bmatrix} E(x) = x^j (x^{i-j} + 1) \end{bmatrix}$$

For undetection:

$$\begin{bmatrix} g(x) \text{ must divide } (x^t + 1) \end{bmatrix}$$

Therefore:

- ✓ A good generator must NOT divide $(x^t + 1)$
- ✓ For all practical t in range

This is why generator selection is non-trivial.

5.3 Detection of Odd Number of Errors

If:

$$\begin{bmatrix} E(x) = \sum x^k \end{bmatrix}$$

with an odd number of terms,

Then:

$$\begin{bmatrix} E(1) = 1 \end{bmatrix}$$

If $g(x)$ contains factor $(x + 1)$, then:

$$\begin{bmatrix} g(1) = 0 \end{bmatrix}$$

Thus:

$g(x)$ cannot divide $E(x)$.

✓ **Therefore:**

If generator contains $(x+1)$,
CRC detects ALL odd number of bit errors.

This is a major theoretical guarantee.

6 Summary of Generator Requirements

Property	Why It Matters
At least two terms	Avoid trivial divisibility

Constant term = 1	Detect single-bit errors
Contains $(x+1)$	Detect all odd-bit errors
Does not divide $(x^t + 1)$	Detect double-bit errors
High degree	Detect longer burst errors

7 Burst Error Detection Strength

If generator degree = r :

- ✓ Detects all burst errors of length $\leq r$
- ✓ Detects most burst errors of length $> r$
- ✓ Probability of undetected random error $\approx (1/2^r)$

Example:

- CRC-16 $\rightarrow$ ~ 1 in 65,536 chance
- CRC-32 $\rightarrow$ ~ 1 in 4 billion chance

8 Case Study: Mathematical Failure Example

Given:

[
 $g(x) = 1001$
]

Which equals:

[
 $x^3 + 1$
]

If error pattern:

$$\begin{bmatrix} E(x) = 1001 \end{bmatrix}$$

Then:

$$\begin{bmatrix} E(x) = g(x) \end{bmatrix}$$

Thus:

$$\begin{bmatrix} E(x) / g(x) = 1 \end{bmatrix}$$

Remainder = 0

Syndrome = 0

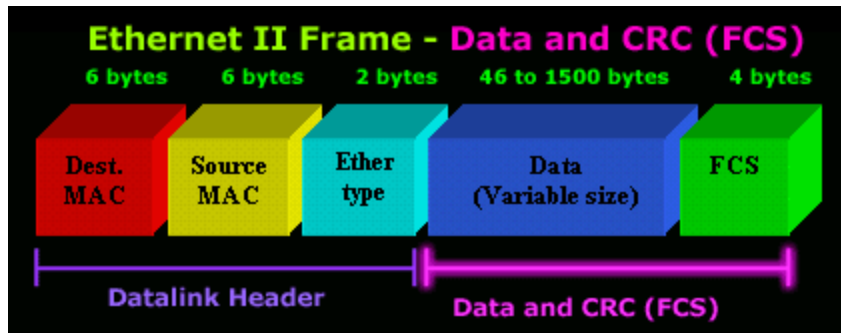
Receiver accepts corrupted data.

This is not a bug.

It is a mathematical inevitability.

9 Why CRC Is Still Extremely Reliable

$$\begin{array}{r}
 4x^3 + 5x^2 + 9x + 12 \quad \overline{) \quad 12x^5 + 23x^4 + 57x^3 + 79x^2 + 69x + 60} \\
 \underline{- 12x^5 + 15x^4 + 27x^3 + 36x^2} \\
 8x^4 + 30x^3 + 43x^2 + 69x \\
 \underline{- 8x^4 + 10x^3 + 18x^2 + 24x} \\
 20x^3 + 25x^2 + 45x + 60 \\
 \underline{- 20x^3 + 25x^2 + 45x + 60} \\
 0
 \end{array}$$



In real systems:

- Generators are very carefully designed
- Degrees are large (16, 32, or more)
- Polynomial properties are chosen after deep mathematical study

Natural noise rarely produces structured errors equal to multiples of the generator.

Thus probability is extremely low.

10 Conceptual Insight: CRC Is a Subspace Check

Cyclic codes form a vector subspace over $GF(2)$.

CRC essentially checks:

"Is this received vector still in the cyclic code subspace?"

If yes → accept

If not → reject

Error detection = membership testing in a structured algebraic space.

11 The Deep Takeaway

CRC fails only when:

[
 $E(x) \in \text{Ideal generated by } g(x)$
]

So generator design defines the "blind spots".

Good generator → extremely few blind spots

Bad generator → many blind spots

